UNITED INTERNATIONAL UNIVERSITY

TERM PAPER

DESIGN AND IMPLEMENTATION OF A PERSONAL INCOME, BUDGET AND EXPENSE TRACKING DATABASE SYSTEM

SURMITTED BY:

Name : Khadiza Rehan

Student ID : 0112420022

SUBMITTED TO:

Md. Golam Hossain

Course Code : CSE 3522

Course Name : Database Management Systems Laboratory

Section : B

Department of Computer Science and Engineering

United International University, UIU.

Submission Date: 04.10.2026



Contents

Abstract 4

1. Introduction 4

1.1 Background 4

1.2 Problem Statement 5

1.3 Objectives 5

1.4 Scope of the System 5

1.5 Significance of the Study 5

2. Literature Review 6

2.1 Personal Financial Management Systems 6

2.2 Relational Database Approach 6

2.3 Database Performance and Transaction Management 6

2.4 Data Warehousing and Star Schema 7

2.5 Business Intelligence Using SQL 7

2.6 Research Gap and Proposed Approach 7

3. Design and Methodology 7

3.1 System Requirements Analysis 7

3.1.1 Functional Requirements 7

3.2 Identification of Entities 8

3.3 Business Rules 8

3.4 Data Model and ERD 9

3.5 Relational Schema 10

3.6 Normalization 13

3.6.1 First Normal Form (1NF) 13

3.6.2 Second Normal Form (2NF) 13

3.6.3 Third Normal Form (3NF) 13

3.6.4 Benefits of Normalization 13

4. Database Implementation 13

4.1 Implementation Environment 13

4.2 Database Creation 14
4.3 Table Creation 15
4.4 Integrity Constraints 16
4.5 Sample Data 17
5. Indexing 18
6. Partitioning 18
7. Transaction Management 19
7.1 COMMIT Operation 19
7.2 ROLLBACK Operation 20
8. Star Schema and BI Design 21
9. Business Intelligence Report Generation 23
9.1 JOIN Query 23
9.2 SQL View 24
9.3 Materialized-View Equivalent 25
9.4 Analytical Reports 25
9.4.1 Category-Wise Expense Report 25
9.4.2 Budget Utilization Report 26
10. Results and Discussion 26
11. Conclusion and Future Scope 27
11.1 Conclusion 27
11.2 Future Scope 27
References 27

Abstract

Personal financial management requires individuals to maintain accurate records of their income, expenses, budgets, account balances, and savings. Managing such information manually or through unstructured records can make financial analysis difficult and may lead to inconsistent or incomplete data. This term paper presents the design and implementation of a Personal Income, Budget and Expense Tracking System using a relational database management approach.

The proposed system provides a structured platform for storing financial accounts, income and expense transactions, transaction categories, budgets, and fund transfers. The proposed system provides a structured platform for storing financial accounts, income and expense transactions, transaction categories, budgets, and fund transfers. A normalized relational data model is designed and implemented using **MariaDB 10.4.32 through the XAMPP environment**.

Database integrity is maintained through primary keys, foreign keys, unique constraints, check constraints, and other validation rules. Indexing and partitioning techniques are incorporated to improve data retrieval and manage growing transaction records. Transaction management is applied to ensure reliable financial operations using commit and rollback mechanisms.

In addition to operational data management, a star schema is designed for analytical processing. Joins, views, and a materialized-view equivalent are used to generate business intelligence reports including monthly income and expenditure, category-wise spending, budget utilization, account balances, and savings analysis.

1. Introduction

1.1 Background

Effective personal financial management requires accurate records of income, expenses, budgets, savings, and financial accounts. As financial transactions increase across different sources and spending categories, maintaining these records manually becomes difficult and may lead to unorganized financial information.

A database-driven system provides a structured solution by storing related financial data in organized tables while supporting data integrity, efficient retrieval, and reliable transaction management. This term paper proposes a Personal Income, Budget and Expense Tracking System implemented using **MariaDB 10.4.32 through XAMPP**. The system uses a relational database for daily financial operations and a dimensional star schema for business intelligence and financial analysis.

1.2 Problem Statement

Personal financial information is often maintained through notebooks, spreadsheets, or separate digital records, which can lead to duplication, incorrect categorization, difficulty in budget monitoring, and inconsistent account balances. These methods also make historical financial analysis more difficult as transaction data grows.

Therefore, a structured database system is needed to manage users, accounts, categories, transactions, budgets, and transfers while maintaining data integrity. The system should also support efficient data retrieval and analytical reporting for better understanding of financial activities.

1.3 Objectives

The primary objective of this project is to design and implement a relational database system for managing personal income, expenses, budgets, and financial accounts.

The specific objectives are:

1. To design a **normalized relational data model** with appropriate integrity constraints.
2. To improve data retrieval using **indexing and partitioning** techniques.
3. To ensure reliable financial operations using **transaction management with COMMIT and ROLLBACK**.
4. To design a **star schema** for analytical processing and business intelligence.
5. To generate financial reports using **SQL joins, views, and a materialized-view equivalent**.
6. To analyze **income, expenses, budgets, savings, and account activities** through BI reports.

1.4 Scope of the System

The proposed system focuses on the database layer of personal financial management. It manages users, multiple financial accounts, income and expense categories, transactions, monthly budgets, category-wise budget allocations, and fund transfers.

The system also supports analytical reports such as **monthly income versus expenses, category-wise spending, budget utilization, account activity, and savings analysis**. The project focuses on database design, optimization, transaction processing, and analytical reporting; development of a complete web or mobile interface is outside its scope.

1.5 Significance of the Study

This project demonstrates the practical application of major database concepts, including **relational modeling, normalization, integrity constraints, indexing, partitioning, transaction management, and business intelligence** in a personal financial management system.

It also combines **operational and analytical database approaches**: the normalized relational schema supports reliable daily financial operations, while the star schema enables efficient aggregation and reporting. Thus, the project demonstrates how transactional and analytical techniques can be integrated within a single database system.

2. Literature Review

2.1 Personal Financial Management Systems

Personal financial management systems help individuals organize and analyze **income, expenses, savings, budgets, and account balances**. Traditional methods such as notebooks and spreadsheets are simple but can become difficult to maintain as financial records increase.

Database-based systems provide a more structured approach by storing related financial information in organized tables. This allows relationships among **users, accounts, categories, transactions, and budgets** to be maintained while supporting efficient retrieval of historical data and financial summaries. [1]

2.2 Relational Database Approach

The relational database model organizes data into related tables, where **primary and foreign keys** establish relationships among entities such as users, accounts, transactions, categories, and budgets.

Normalization reduces data redundancy and prevents insertion, update, and deletion anomalies by separating financial information into appropriate tables. **Integrity constraints**, including primary keys, foreign keys, unique constraints, and check constraints, further ensure that stored financial data remains valid, consistent, and properly related. [1], [2]

2.3 Database Performance and Transaction Management

As financial records grow, **indexing and partitioning** help maintain efficient database performance. Indexes can improve frequently used searches, such as retrieving transactions by account and date, while partitioning can organize large historical transaction datasets based on time periods.

Transaction management is essential for financial operations involving multiple database updates. For example, a fund transfer requires both deduction from the source account and addition to the destination account. **COMMIT and ROLLBACK** help ensure that such operations are completed consistently or cancelled when necessary. [2], [5]

2.4 Data Warehousing and Star Schema

Operational databases support regular data insertion, modification, and retrieval, while analytical processing often requires aggregation of historical data. **Dimensional modeling** provides a suitable structure for such analysis.

A **star schema** consists of a central fact table connected to multiple dimension tables. In this system, financial transactions are stored in a fact table and analyzed using **date, user, account, and category dimensions**, supporting reports such as monthly spending trends, category-wise expenses, income-versus-expense comparisons, and account activity. [3]

2.5 Business Intelligence Using SQL

Business intelligence (BI) transforms stored data into meaningful information for analysis and decision-making. SQL supports BI through techniques such as **JOINs, views, and precomputed summaries**, which combine and organize data for efficient analysis.

In this system, these techniques are used to generate reports on **monthly income and expenses, category-wise spending, budget utilization, account activity, and savings trends**. [2]

2.6 Research Gap and Proposed Approach

Many basic personal finance systems mainly focus on transaction recording and simple financial summaries. This project adopts a broader database-oriented approach by combining both **operational and analytical database concepts**.

The proposed system integrates **normalized relational modeling, integrity constraints, indexing, partitioning, transaction management, dimensional modeling, and BI reporting**, demonstrating how these techniques can work together within a complete personal financial management database.

3. Design and Methodology

3.1 System Requirements Analysis

The **Personal Income, Budget and Expense Tracking System** is designed to manage individual financial activities in a structured relational database. Functional requirements and business rules were identified to determine the required data, relationships among entities, and restrictions necessary to maintain data integrity.

3.1.1 Functional Requirements

The major functional requirements are:

- Support multiple users with individual financial records.
- Maintain multiple accounts such as **cash, bank, and mobile banking** accounts.

- Record and categorize both **income and expense transactions**.
- Associate each transaction with a valid **account and category**.
- Create **monthly budgets** with category-wise spending limits.
- Support **fund transfers** between financial accounts.
- Maintain historical transaction data for financial analysis.
- Generate analytical reports on **income, expenses, budgets, account activity, and savings**.

3.2 Identification of Entities

Based on the functional requirements, seven major entities were identified: **User, Account, Category, Financial Transaction, Budget, Budget Detail, and Transfer**. These entities represent the core financial data of the system and are connected through defined relationships to form the operational database.

Entity	Description
User	Stores information about each registered user.
Account	Represents financial accounts owned by a user, such as cash, bank, or mobile banking accounts.
Category	Classifies financial transactions as different types of income or expenses.
Financial Transaction	Records individual income and expense activities.
Budget	Represents a user's financial budget for a specific month and year.
Budget Detail	Stores category-specific spending limits associated with a budget.
Transfer	Records the movement of funds between two financial accounts.

Table 1: Major Entities of the Proposed System

3.3 Business Rules

The following business rules define the main relationships and restrictions of the database:

1. A user can own multiple **accounts, categories, and monthly budgets**.
2. Each financial transaction belongs to one **account and one category**, while an account or category can be associated with multiple transactions.

3. Each budget can contain multiple **category-wise budget details**, but the same category cannot appear more than once in a budget.
4. A fund transfer must occur between **two different accounts belonging to the same user**.
5. All financial amounts must be **greater than zero**.
6. Categories must be classified as either **INCOME or EXPENSE**.
7. A user can have only **one budget for a specific month and year**.

3.4 Data Model and ERD

The data model was developed based on the identified functional requirements and business rules. It consists of seven major entities: **User, Account, Category, Financial Transaction, Budget, Budget Detail, and Transfer**. Primary keys uniquely identify records, while foreign keys establish relationships among the entities. The attributes of these entities are presented in **Table 2**.

Entity	Attributes
User	user_id (PK), full_name, email, password, created_at
Account	account_id (PK), user_id (FK), account_name, account_type, balance, created_at
Category	category_id (PK), user_id (FK), category_name, category_type
Financial Transaction	transaction_id (PK), account_id (FK), category_id (FK), amount, transaction_date, description
Budget	budget_id (PK), user_id (FK), budget_month, budget_year, created_at
Budget Detail	budget_detail_id (PK), budget_id (FK), category_id (FK), allocated_amount
Transfer	transfer_id (PK), user_id (FK), source_account_id (FK), destination_account_id (FK), amount, transfer_date, description

Table 2: Entities and Attributes of the Operational Database

The **User** entity owns financial information, including accounts, categories, and budgets. **Account** and **Category** entities support financial transactions, while **Budget** and **Budget Detail** manage monthly financial planning. The **Transfer** entity records fund movements between source and destination accounts.

The main relationships and cardinalities are:

1. **User → Account, Category, Budget, Transfer:** 1:M

2. **Account → Financial Transaction:** 1:M
3. **Category → Financial Transaction:** 1:M
4. **Budget → Budget Detail:** 1:M
5. **Category → Budget Detail:** 1:M
6. **Account → Transfer:** 1:M as both source and destination accounts.

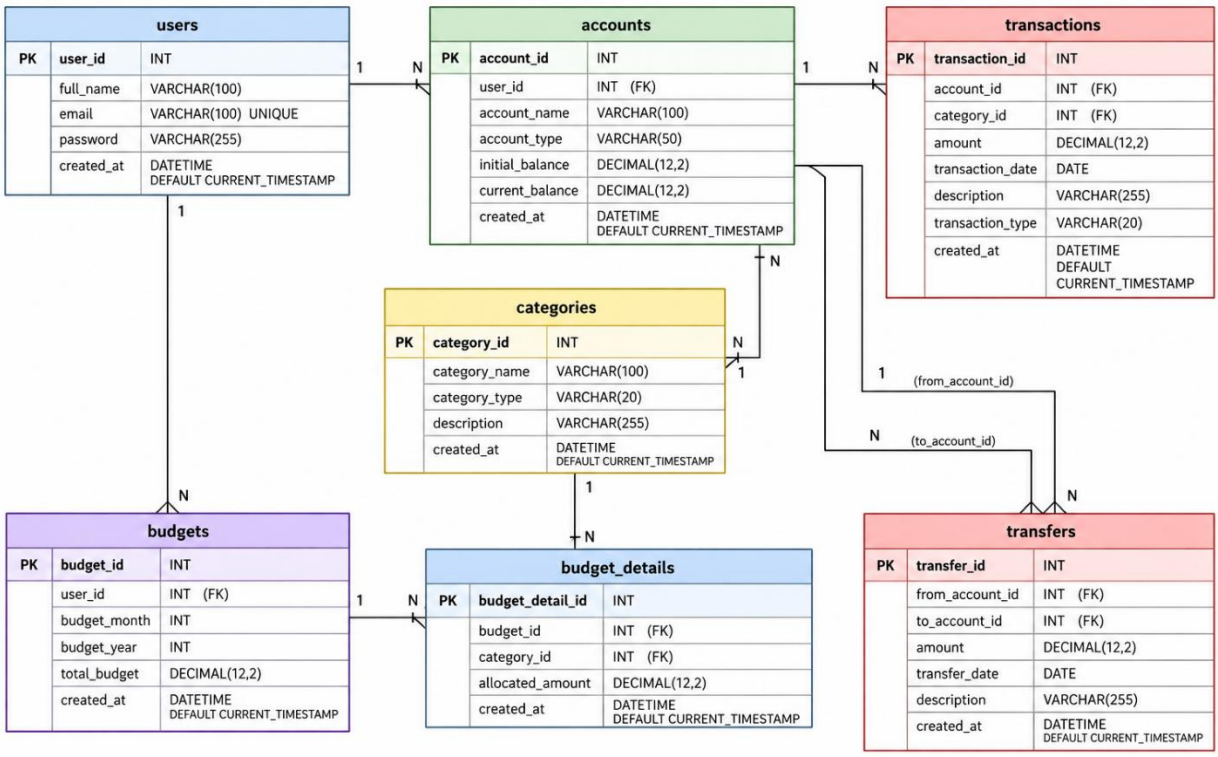


Figure 4: Entity–Relationship Diagram (ERD) of the Database System

The ERD shows the logical relationships among the seven operational entities. **Foreign keys** connect related entities, helping reduce data redundancy and maintain consistent relationships among financial records.

3.5 Relational Schema

The conceptual data model was converted into a **relational schema**, where each entity is represented as a relation. **Primary keys** uniquely identify records, while **foreign keys** maintain relationships between related relations. The relational schema is defined as follows:

USER(

user_id PK,

full_name,

email,

password,

created_at

)

ACCOUNT(

account_id PK,

user_id FK → USER.user_id,

account_name,

account_type,

balance,

created_at

)

CATEGORY(

category_id PK,

user_id FK → USER.user_id,

category_name,

category_type

)

TRANSACTION(

transaction_id PK,

account_id FK → ACCOUNT.account_id,

category_id FK → CATEGORY.category_id,

amount,

transaction_date,

description

)

BUDGET(

budget_id PK,

user_id FK → USER.user_id,

budget_month,

budget_year,

created_at

)

BUDGET_DETAIL(

budget_detail_id PK,

budget_id FK → BUDGET.budget_id,

category_id FK → CATEGORY.category_id,

allocated_amount

)

TRANSFER(

transfer_id PK,

user_id FK → USER.user_id,

source_account_id FK → ACCOUNT.account_id,

destination_account_id FK → ACCOUNT.account_id,

amount,

transfer_date,

description

)

The relational schema maintains the ERD relationships through **foreign keys**, while additional **unique and validation constraints** enforce the defined business rules.

3.6 Normalization

Normalization was applied to reduce **data redundancy** and prevent **insertion, update, and deletion anomalies**. The operational database was designed to satisfy **1NF, 2NF, and 3NF**.

3.6.1 First Normal Form (1NF)

A relation satisfies **1NF** when attributes contain atomic values and no repeating groups exist. In this system, each attribute stores a single value, and related information such as accounts and transactions is stored separately. Therefore, the relations satisfy **1NF**.

3.6.2 Second Normal Form (2NF)

A relation satisfies **2NF** when it is in 1NF and all non-key attributes fully depend on the primary key. The operational tables use primary keys such as user_id, account_id, and transaction_id, with their non-key attributes fully dependent on the corresponding key. Therefore, the relations satisfy **2NF**.

3.6.3 Third Normal Form (3NF)

A relation satisfies **3NF** when it is in 2NF and contains no inappropriate transitive dependencies. User, account, category, transaction, and budget information are stored in separate related tables rather than being repeatedly stored together. Therefore, the operational schema satisfies **3NF**.

3.6.4 Benefits of Normalization

Normalization provides the following benefits:

Reduces data redundancy.

Prevents insertion, update, and deletion anomalies.

Improves data consistency.

Separates different business entities clearly.

Supports referential integrity through foreign keys.

4. Database Implementation

4.1 Implementation Environment

The database system was implemented using **MariaDB 10.4.32** through the **XAMPP** local server environment. The **MariaDB Command-Line Client (CMD)** was used to execute SQL statements and verify database results. The environment supports the required features, including

relational tables, integrity constraints, indexing, views, and transaction management. The implementation environment is summarized below:

Component	Technology
Database Management System	MariaDB 10.4.32
Local Server Environment	XAMPP
Database Interface	MariaDB Command-Line Client (CMD)
Query Language	SQL
Storage Engine	InnoDB

Table 3: Database Implementation Environment

The database server version was verified through the command-line client, confirming that **MariaDB 10.4.32** was running successfully in the **XAMPP environment.**

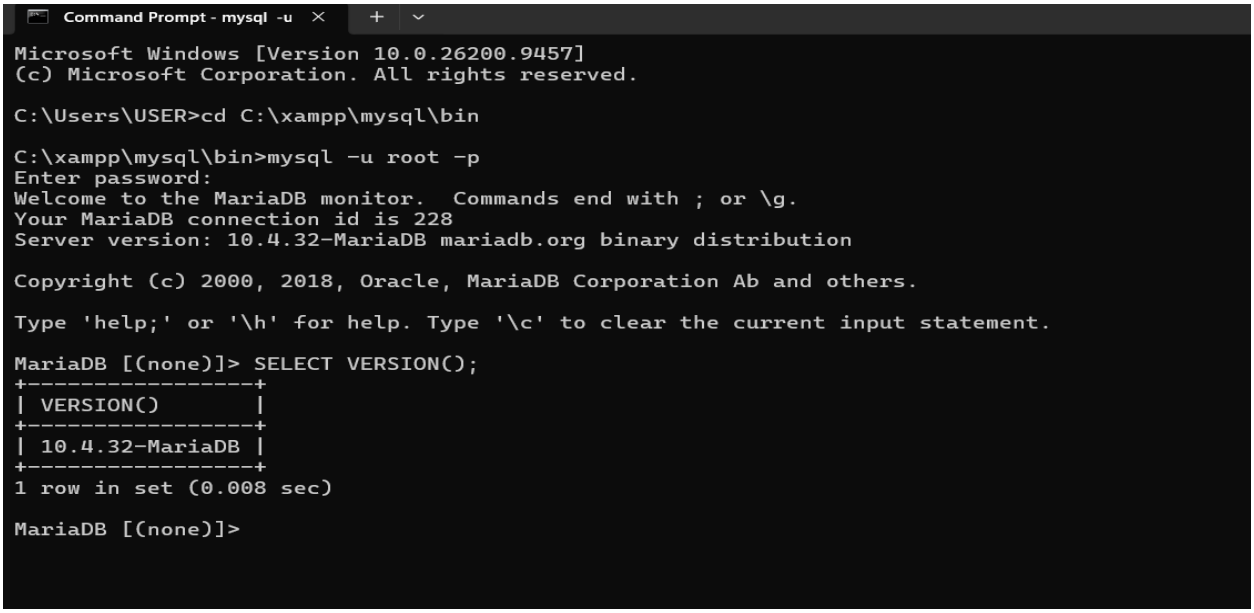


Figure 2: Verification of the MariaDB Database Environment

4.2 Database Creation

Before implementing the relational tables, a dedicated database named **personal_finance_db** was created and selected as the active database. It serves as the logical container for all tables and other database objects of the proposed system.

```
MariaDB [(none)]> CREATE DATABASE personal_finance_db;
Query OK, 1 row affected (0.015 sec)

MariaDB [(none)]> USE personal_finance_db;
Database changed
MariaDB [personal_finance_db]> SELECT DATABASE();
+--------------------+
| DATABASE()         |
+--------------------+
| personal_finance_db |
+--------------------+
1 row in set (0.001 sec)

MariaDB [personal_finance_db]> SHOW TABLES;
Empty set (0.010 sec)

MariaDB [personal_finance_db]>
```

Figure 3: Creation and Selection of the Personal Finance Database

The result confirms that **personal_finance_db** was successfully created and selected. The empty table list shows the database state before the relational tables were implemented.

4.3 Table Creation

The relational schema was implemented using seven **InnoDB** tables: users, accounts, categories, transactions, budgets, budget_details, and transfers. The tables were created in dependency order with appropriate **primary keys, foreign keys, unique constraints, check constraints, and default values** to maintain data integrity. The SHOW TABLES command was used to verify their successful creation.

```
MariaDB [personal_finance_db]> SHOW TABLES;
+-------------------------------+
| Tables_in_personal_finance_db |
+-------------------------------+
| accounts                      |
| budget_details                |
| budgets                       |
| categories                    |
| transactions                  |
| transfers                     |
| users                         |
+-------------------------------+
7 rows in set (0.002 sec)

MariaDB [personal_finance_db]>
```

Figure 4: Operational Tables Created in the Personal Finance Database

4.4 Integrity Constraints

Integrity constraints were applied to maintain valid and consistent financial data. **Primary keys** uniquely identify records, **foreign keys** maintain table relationships, **unique constraints** prevent duplicate entries, and **check constraints** restrict invalid values. For example, budget_details prevents duplicate category allocations within the same budget and ensures that allocated amounts are positive.

Constraint Type	**Example**	**Purpose**
Primary Key	user_id, account_id, transaction_id	Uniquely identifies each record
Foreign Key	accounts.user_id → users.user_id	Maintains referential integrity
Unique Key	(user_id, budget_month, budget_year)	Prevents duplicate monthly budgets
Unique Key	(budget_id, category_id)	Prevents duplicate category allocation
Check	amount > 0	Prevents invalid financial amounts
Check	category_typeIN ('INCOME','EXPENSE')	Restricts category type
Check	source_account_id <> destination_account_id	Prevents transfer to the same account
NOT NULL	amount, transaction_date, etc.	Prevents required data from being omitted

Table 4: Major Integrity Constraints Used in the Database

```
MariaDB [personal_finance_db]> SHOW CREATE TABLE budget_details\G
*************************** 1. row ***************************
       Table: budget_details
Create Table: CREATE TABLE `budget_details` (
  `budget_detail_id` int(11) NOT NULL AUTO_INCREMENT,
  `budget_id` int(11) NOT NULL,
  `category_id` int(11) NOT NULL,
  `allocated_amount` decimal(12,2) NOT NULL,
  PRIMARY KEY (`budget_detail_id`),
  UNIQUE KEY `uq_budget_category` (`budget_id`,`category_id`),
  KEY `fk_budget_details_category` (`category_id`),
  CONSTRAINT `fk_budget_details_budget` FOREIGN KEY (`budget_id`) REFERENCES
 `budgets` (`budget_id`) ON DELETE CASCADE,
  CONSTRAINT `fk_budget_details_category` FOREIGN KEY (`category_id`) REFERE
NCES `categories` (`category_id`),
  CONSTRAINT `chk_allocated_amount` CHECK (`allocated_amount` > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
1 row in set (0.004 sec)

MariaDB [personal_finance_db]>
```

```
MariaDB [personal_finance_db]> SHOW CREATE TABLE transfers\G
*************************** 1. row ***************************
      Table: transfers
Create Table: CREATE TABLE `transfers` (
  `transfer_id` int(11) NOT NULL AUTO_INCREMENT,
  `user_id` int(11) NOT NULL,
  `source_account_id` int(11) NOT NULL,
  `destination_account_id` int(11) NOT NULL,
  `amount` decimal(12,2) NOT NULL,
  `transfer_date` date NOT NULL,
  `description` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`transfer_id`),
  KEY `fk_transfers_user` (`user_id`),
  KEY `fk_transfer_source` (`source_account_id`),
  KEY `fk_transfer_destination` (`destination_account_id`),
  CONSTRAINT `fk_transfer_destination` FOREIGN KEY (`destination_account_id`) REFERENCES `accounts` (`account_id`),
  CONSTRAINT `fk_transfer_source` FOREIGN KEY (`source_account_id`) REFERENCES `accounts` (`account_id`),
  CONSTRAINT `fk_transfers_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`user_id`),
  CONSTRAINT `chk_transfer_amount` CHECK (`amount` > 0),
  CONSTRAINT `chk_different_accounts` CHECK (`source_account_id` <> `destination_account_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
1 row in set (0.012 sec)

MariaDB [personal_finance_db]>
```

Figure 5: Implementation of Foreign Key, Unique, and Check Constraints

4.5 Sample Data

Sample data was inserted to test the database **relationships, constraints, transactions, and analytical queries**. The dataset includes users, accounts, income and expense categories, financial transactions, budgets, and fund transfers

```
MariaDB [personal_finance_db]> SELECT * FROM transactions;
+----------------+------------+-------------+----------+------------------+------------------------+
| transaction_id | account_id | category_id | amount   | transaction_date | description            |
+----------------+------------+-------------+----------+------------------+------------------------+
|              1 |          2 |           1 | 50000.00 | 2026-07-01       | Monthly salary         |
|              2 |          2 |           3 |  4500.00 | 2026-07-05       | Groceries              |
|              3 |          1 |           4 |  2000.00 | 2026-07-10       | Transportation         |
|              4 |          2 |           5 |  6000.00 | 2026-07-15       | Course fee             |
|              5 |          3 |           6 |  2500.00 | 2026-07-20       | Internet and utilities |
|              6 |          2 |           2 | 12000.00 | 2026-08-03       | Freelance project      |
|              7 |          1 |           3 |  5000.00 | 2026-08-08       | Food expenses          |
|              8 |          3 |           7 |  3000.00 | 2026-08-15       | Entertainment          |
|              9 |          4 |           8 | 40000.00 | 2026-08-01       | Monthly salary         |
|             10 |          4 |           9 |  4000.00 | 2026-08-12       | Groceries              |
+----------------+------------+-------------+----------+------------------+------------------------+
10 rows in set (0.002 sec)

MariaDB [personal_finance_db]>
```

Figure 6: Sample Transaction Data Stored in the Database

5. Indexing

Indexes were created on frequently searched transaction attributes to improve query performance. A composite index on account and transaction date supports efficient retrieval of account-wise transaction histories.

```
MariaDB [personal_finance_db]> EXPLAIN
    -> SELECT *
    -> FROM transactions
    -> WHERE account_id = 2
    -> AND transaction_date BETWEEN '2026-07-01' AND '2026-08-31';
+------+-------------+--------------+------+-------------------+------+---------+------+------+-------------+
| id   | select_type | table        | type | possible_keys     | key  | key_len | ref  | rows | Extra       |
+------+-------------+--------------+------+-------------------+------+---------+------+------+-------------+
|    1 | SIMPLE      | transactions | ALL  | idx_account_date | NULL | NULL    | NULL | 10   | Using where |
+------+-------------+--------------+------+-------------------+------+---------+------+------+-------------+
1 row in set (0.001 sec)

MariaDB [personal_finance_db]>
```

Figure 7: Query Analysis Using the Composite Transaction Index

The composite index supports filtering by account and transaction date, which are common attributes in financial-history queries.

6. Partitioning

Partitioning was demonstrated on a historical transaction table using transaction year as the partitioning criterion. Year-based partitions allow large historical datasets to be logically divided by time.

```
MariaDB [personal_finance_db]> SELECT
    ->     PARTITION_NAME,
    ->     PARTITION_METHOD,
    ->     PARTITION_EXPRESSION,
    ->     TABLE_ROWS
    -> FROM information_schema.PARTITIONS
    -> WHERE TABLE_SCHEMA = 'personal_finance_db'
    -> AND TABLE_NAME = 'transaction_history';
+----------------+------------------+--------------------------+------------+
| PARTITION_NAME | PARTITION_METHOD | PARTITION_EXPRESSION     | TABLE_ROWS |
+----------------+------------------+--------------------------+------------+
| p2025          | RANGE            | year(`transaction_date`) |          0 |
| p2026          | RANGE            | year(`transaction_date`) |         10 |
| p2027          | RANGE            | year(`transaction_date`) |          0 |
| p_future       | RANGE            | year(`transaction_date`) |          0 |
+----------------+------------------+--------------------------+------------+
4 rows in set (0.037 sec)

MariaDB [personal_finance_db]>
```

Figure 8: Year-Based Partitioning of Historical Transaction Data

The historical records are divided into yearly partitions, demonstrating time-based partition management for growing financial data.

7. Transaction Management

Transaction management ensures that related financial operations are completed as a single logical unit. For fund transfers, account balances are updated together so that partial updates cannot leave the database inconsistent. COMMIT confirms a successful transaction, while ROLLBACK cancels an incomplete or invalid operation.

7.1 COMMIT Operation

A successful transfer was performed from the Bank Account to the Mobile Banking account using a database transaction.

```
MariaDB [personal_finance_db]> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

MariaDB [personal_finance_db]>
MariaDB [personal_finance_db]> UPDATE accounts
    -> SET balance = balance - 5000
    -> WHERE account_id = 2;
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [personal_finance_db]>
MariaDB [personal_finance_db]> UPDATE accounts
    -> SET balance = balance + 5000
    -> WHERE account_id = 3;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [personal_finance_db]>
MariaDB [personal_finance_db]> INSERT INTO transfers
    -> (user_id, source_account_id, destination_account_id,
    ->  amount, transfer_date, description)
    -> VALUES
    -> (1, 2, 3, 5000, '2026-08-20',
    ->  'Bank to mobile transfer');
Query OK, 1 row affected (0.002 sec)

MariaDB [personal_finance_db]>
MariaDB [personal_finance_db]> COMMIT;
Query OK, 0 rows affected (0.002 sec)

MariaDB [personal_finance_db]> SELECT account_id, account_name, balance
    -> FROM accounts
    -> WHERE account_id IN (2, 3);
+------------+----------------+----------+
| account_id | account_name   | balance  |
+------------+----------------+----------+
|          2 | Bank Account   | 45000.00 |
|          3 | Mobile Banking | 20000.00 |
+------------+----------------+----------+
```

Figure 9: Successful Fund Transfer Using COMMIT

The result confirms that both account balances were updated successfully and the transaction was permanently committed.

7.2 ROLLBACK Operation

ROLLBACK was tested to demonstrate recovery from an unsuccessful or cancelled financial operation.

```
MariaDB [personal_finance_db]> START TRANSACTION;
Query OK, 0 rows affected (0.003 sec)

MariaDB [personal_finance_db]>
MariaDB [personal_finance_db]> UPDATE accounts
    -> SET balance = balance - 3000
    -> WHERE account_id = 2;
Query OK, 1 row affected (0.010 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [personal_finance_db]>
MariaDB [personal_finance_db]> UPDATE accounts
    -> SET balance = balance + 3000
    -> WHERE account_id = 3;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [personal_finance_db]>
MariaDB [personal_finance_db]> ROLLBACK;
Query OK, 0 rows affected (0.009 sec)

MariaDB [personal_finance_db]> SELECT account_id, account_name, balance
    -> FROM accounts
    -> WHERE account_id IN (2, 3);
+------------+----------------+----------+
| account_id | account_name   | balance  |
+------------+----------------+----------+
|          2 | Bank Account   | 45000.00 |
|          3 | Mobile Banking | 20000.00 |
+------------+----------------+----------+
2 rows in set (0.001 sec)
```

Figure 10: Transaction Recovery Using ROLLBACK

The unchanged balances verify that the cancelled transaction did not permanently modify the database.

8. Star Schema and BI Design

A star schema was designed for analytical reporting. The central fact_transactions table stores financial measures, while dim_date, dim_user, dim_account, and dim_category provide descriptive dimensions for analysis.

Star Schema Structure

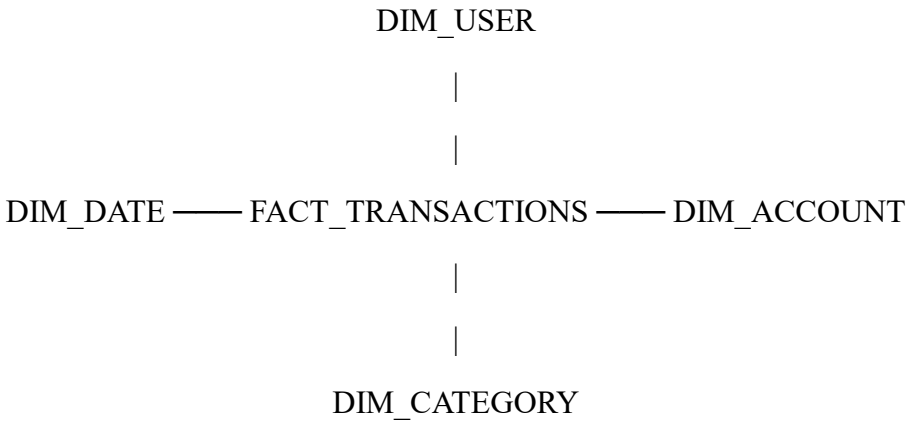


Figure 11: Star Schema for Financial Transaction Analysis

Central fact table:

FACT_TRANSACTIONS

- transaction_key (PK)
- date_key (FK)
- user_key (FK)
- account_key (FK)
- category_key (FK)
- amount

Dimensions:

DIM_DATE — date, day, month, year
DIM_USER — user information
DIM_ACCOUNT — account information
DIM_CATEGORY — income/expense category information

```
MariaDB [personal_finance_db]> SELECT
    ->     f.transaction_key,
    ->     d.full_date,
    ->     u.full_name,
    ->     a.account_name,
    ->     c.category_name,
    ->     c.category_type,
    ->     f.amount
    -> FROM fact_transactions f
    -> JOIN dim_date d
    ->     ON f.date_key = d.date_key
    -> JOIN dim_user u
    ->     ON f.user_key = u.user_key
    -> JOIN dim_account a
    ->     ON f.account_key = a.account_key
    -> JOIN dim_category c
    ->     ON f.category_key = c.category_key;
+-----------------+------------+--------------+---------------+----------------+--------------+----------+
| transaction_key | full_date  | full_name    | account_name  | category_name  | category_type | amount   |
+-----------------+------------+--------------+---------------+----------------+--------------+----------+
|               1 | 2026-07-01 | Khadiza Rehan | Bank Account  | Salary         | INCOME       | 50000.00 |
|               2 | 2026-07-05 | Khadiza Rehan | Bank Account  | Food           | EXPENSE      |  4500.00 |
|               3 | 2026-07-10 | Khadiza Rehan | Cash Wallet   | Transportation | EXPENSE      |  2000.00 |
|               4 | 2026-07-15 | Khadiza Rehan | Bank Account  | Education      | EXPENSE      |  6000.00 |
|               5 | 2026-07-20 | Khadiza Rehan | Mobile Banking | Utilities      | EXPENSE      |  2500.00 |
|               6 | 2026-08-03 | Khadiza Rehan | Bank Account  | Freelancing    | INCOME       | 12000.00 |
|               7 | 2026-08-08 | Khadiza Rehan | Cash Wallet   | Food           | EXPENSE      |  5000.00 |
|               8 | 2026-08-15 | Khadiza Rehan | Mobile Banking | Entertainment  | EXPENSE      |  3000.00 |
|               9 | 2026-08-01 | Nusrat Jahan  | Personal Bank | Salary         | INCOME       | 40000.00 |
|              10 | 2026-08-12 | Nusrat Jahan  | Personal Bank | Food           | EXPENSE      |  4000.00 |
+-----------------+------------+--------------+---------------+----------------+--------------+----------+
10 rows in set (0.001 sec)

MariaDB [personal_finance_db]>
```

Figure 12: Analytical Data Generated from the Star Schema

The result demonstrates how the fact table and dimensions combine to support multidimensional financial analysis.

9. Business Intelligence Report Generation

Business intelligence reports were generated using **JOINs, views, aggregations, and summarized data**. These reports transform stored financial records into useful information about income, expenses, budgets, and savings.

9.1 JOIN Query

A JOIN query was used to combine transaction, account, and category information. This provides a complete view of each transaction with its associated account and financial category.

```
MariaDB [personal_finance_db]> SELECT
    ->     t.transaction_id,
    ->     t.transaction_date,
    ->     a.account_name,
    ->     c.category_name,
    ->     c.category_type,
    ->     t.amount
    -> FROM transactions t
    -> JOIN accounts a
    ->     ON t.account_id = a.account_id
    -> JOIN categories c
    ->     ON t.category_id = c.category_id
    -> ORDER BY t.transaction_date;
+----------------+------------------+----------------+----------------+---------------+----------+
| transaction_id | transaction_date | account_name   | category_name  | category_type | amount   |
+----------------+------------------+----------------+----------------+---------------+----------+
|              1 | 2026-07-01       | Bank Account   | Salary         | INCOME        | 50000.00 |
|              2 | 2026-07-05       | Bank Account   | Food           | EXPENSE       |  4500.00 |
|              3 | 2026-07-10       | Cash Wallet    | Transportation | EXPENSE       |  2000.00 |
|              4 | 2026-07-15       | Bank Account   | Education      | EXPENSE       |  6000.00 |
|              5 | 2026-07-20       | Mobile Banking | Utilities      | EXPENSE       |  2500.00 |
|              9 | 2026-08-01       | Personal Bank  | Salary         | INCOME        | 40000.00 |
|              6 | 2026-08-03       | Bank Account   | Freelancing    | INCOME        | 12000.00 |
|              7 | 2026-08-08       | Cash Wallet    | Food           | EXPENSE       |  5000.00 |
|             10 | 2026-08-12       | Personal Bank  | Food           | EXPENSE       |  4000.00 |
|              8 | 2026-08-15       | Mobile Banking | Entertainment  | EXPENSE       |  3000.00 |
+----------------+------------------+----------------+----------------+---------------+----------+
10 rows in set (0.019 sec)

MariaDB [personal_finance_db]>
```

Figure 13: Financial Transaction Report Using JOIN

9.2 SQL View

A view was created to simplify repeated access to transaction details. It combines transaction, account, category, and user information into a reusable virtual table.

```
MariaDB [personal_finance_db]> CREATE VIEW vw_transaction_details AS
    -> SELECT
    ->     t.transaction_id,
    ->     u.full_name,
    ->     a.account_name,
    ->     c.category_name,
    ->     c.category_type,
    ->     t.amount,
    ->     t.transaction_date
    -> FROM transactions t
    -> JOIN accounts a
    ->     ON t.account_id = a.account_id
    -> JOIN users u
    ->     ON a.user_id = u.user_id
    -> JOIN categories c
    ->     ON t.category_id = c.category_id;
Query OK, 0 rows affected (0.018 sec)

MariaDB [personal_finance_db]> SELECT * FROM vw_transaction_details;
+----------------+---------------+----------------+----------------+---------------+----------+------------------+
| transaction_id | full_name     | account_name   | category_name  | category_type | amount   | transaction_date |
+----------------+---------------+----------------+----------------+---------------+----------+------------------+
|              1 | Khadiza Rehan | Bank Account   | Salary         | INCOME        | 50000.00 | 2026-07-01       |
|              2 | Khadiza Rehan | Bank Account   | Food           | EXPENSE       |  4500.00 | 2026-07-05       |
|              4 | Khadiza Rehan | Bank Account   | Education      | EXPENSE       |  6000.00 | 2026-07-15       |
|              6 | Khadiza Rehan | Bank Account   | Freelancing    | INCOME        | 12000.00 | 2026-08-03       |
|              3 | Khadiza Rehan | Cash Wallet    | Transportation | EXPENSE       |  2000.00 | 2026-07-10       |
|              7 | Khadiza Rehan | Cash Wallet    | Food           | EXPENSE       |  5000.00 | 2026-08-08       |
|              5 | Khadiza Rehan | Mobile Banking | Utilities      | EXPENSE       |  2500.00 | 2026-07-20       |
|              8 | Khadiza Rehan | Mobile Banking | Entertainment  | EXPENSE       |  3000.00 | 2026-08-15       |
|              9 | Nusrat Jahan  | Personal Bank  | Salary         | INCOME        | 40000.00 | 2026-08-01       |
|             10 | Nusrat Jahan  | Personal Bank  | Food           | EXPENSE       |  4000.00 | 2026-08-12       |
+----------------+---------------+----------------+----------------+---------------+----------+------------------+
10 rows in set (0.010 sec)
```

Figure 14: Transaction Details Generated Using SQL View

The view provides simplified access to frequently required financial transaction information.

9.3 Materialized-View Equivalent

MariaDB does not provide native materialized views. Therefore, a **materialized-view equivalent** was implemented using a physical summary table that stores precomputed monthly financial results. This reduces the need to repeatedly calculate the same analytical aggregation.

```
MariaDB [personal_finance_db]> SELECT * FROM mv_monthly_finance_summary;
+---------+---------+----------+--------------+---------------+----------+
| user_id | year_no | month_no | total_income | total_expense | savings  |
+---------+---------+----------+--------------+---------------+----------+
|       1 |    2026 |        7 |     50000.00 |      15000.00 | 35000.00 |
|       1 |    2026 |        8 |     12000.00 |       8000.00 |  4000.00 |
|       2 |    2026 |        8 |     40000.00 |       4000.00 | 36000.00 |
+---------+---------+----------+--------------+---------------+----------+
3 rows in set (0.001 sec)

MariaDB [personal_finance_db]>
```

Figure 15: Precomputed Monthly Financial Summary Using Materialized-View Equivalent

The summary table stores precomputed monthly income, expenses, and savings for efficient analytical reporting**. [6]**

9.4 Analytical Reports

9.4.1 Category-Wise Expense Report

Category-wise expenditure was analyzed to identify how total expenses are distributed among different spending categories.

```
MariaDB [personal_finance_db]> SELECT
    ->     c.category_name,
    ->     SUM(t.amount) AS total_expense
    -> FROM transactions t
    -> JOIN categories c
    ->     ON t.category_id = c.category_id
    -> WHERE c.category_type = 'EXPENSE'
    -> GROUP BY c.category_name
    -> ORDER BY total_expense DESC;
+----------------+---------------+
| category_name  | total_expense |
+----------------+---------------+
| Food           |      13500.00 |
| Education      |       6000.00 |
| Entertainment  |       3000.00 |
| Utilities      |       2500.00 |
| Transportation |       2000.00 |
+----------------+---------------+
5 rows in set (0.003 sec)

MariaDB [personal_finance_db]>
```

Figure 16: Category-Wise Expense Analysis

9.4.2 Budget Utilization Report

Budget utilization was analyzed by comparing the allocated budget of each category with its actual expenditure.

```
MariaDB [personal_finance_db]> SELECT
    ->     c.category_name,
    ->     bd.allocated_amount,
    ->     COALESCE(SUM(t.amount), 0) AS actual_expense,
    ->     bd.allocated_amount - COALESCE(SUM(t.amount), 0)
    ->         AS remaining_budget
    -> FROM budget_details bd
    -> JOIN budgets b
    ->     ON bd.budget_id = b.budget_id
    -> JOIN categories c
    ->     ON bd.category_id = c.category_id
    -> LEFT JOIN transactions t
    ->     ON t.category_id = c.category_id
    ->     AND MONTH(t.transaction_date) = b.budget_month
    ->     AND YEAR(t.transaction_date) = b.budget_year
    ->     AND t.account_id IN (
    ->         SELECT account_id
    ->         FROM accounts
    ->         WHERE user_id = b.user_id
    ->     )
    -> GROUP BY
    ->     bd.budget_detail_id,
    ->     c.category_name,
    ->     bd.allocated_amount
    -> ORDER BY c.category_name;
+---------------+------------------+----------------+------------------+
| category_name | allocated_amount | actual_expense | remaining_budget |
+---------------+------------------+----------------+------------------+
| Education     |          7000.00 |        6000.00 |          1000.00 |
| Entertainment |          4000.00 |        3000.00 |          1000.00 |
| Food          |          6000.00 |        4000.00 |          2000.00 |
| Food          |          6000.00 |        4500.00 |          1500.00 |
| Food          |          7000.00 |        5000.00 |          2000.00 |
| Transportation |         3000.00 |        2000.00 |          1000.00 |
| Transportation |         3500.00 |           0.00 |          3500.00 |
| Utilities     |          3000.00 |        2500.00 |           500.00 |
+---------------+------------------+----------------+------------------+
8 rows in set (0.012 sec)

MariaDB [personal_finance_db]>
```

Figure 17: Budget Allocation and Actual Expense Analysis

The report compares planned and actual expenditure, allowing the remaining budget for each category to be evaluated.

10. Results and Discussion

The proposed Personal Income, Budget and Expense Tracking System was successfully implemented using **MariaDB 10.4.32**. The normalized relational schema maintained structured financial data, while integrity constraints helped prevent invalid and inconsistent records.

The implementation demonstrated **indexing, partitioning, transaction management, and star schema design**. COMMIT and ROLLBACK supported reliable fund-transfer operations, while JOINs, views, and the materialized-view equivalent supported analytical processing. The generated BI reports presented **income, expenses, category-wise spending, budget utilization, and savings information**.

Overall, the results demonstrate that the proposed database can support both **operational financial management and analytical reporting** within a single system.

11. Conclusion and Future Scope

11.1 Conclusion

This project designed and implemented a complete database solution for personal financial management. The system integrates **relational modeling, normalization, integrity constraints, indexing, partitioning, transaction management, dimensional modeling, and BI reporting**.

The implementation demonstrates how a structured database can reliably manage daily financial records while also supporting meaningful financial analysis through a star schema, SQL queries, views, and summarized data.

11.2 Future Scope

The system can be extended with a **web or mobile interface, user authentication, automated transaction import, recurring transactions, and graphical dashboards**. Future versions may also include advanced financial forecasting and automated budget recommendations.

References

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[3] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Wiley, 2013.

[4] MariaDB, "MariaDB Server Documentation," MariaDB Documentation. MariaDB Server Documentation

[5] MariaDB, "Atomic Compound Statements," *MariaDB*, 2017. MariaDB Transaction and Rollback Reference

[6] MariaDB, "Migration Guide from PostgreSQL to MariaDB," *MariaDB Migration Guide*. MariaDB Migration Guide